

Especificación Técnica y Funcional — AppHR

Sistema de Gestión y Optimización en la Asignación de Habitaciones Hoteleras. Multi-tenant (SaaS por hotel) con arquitectura **frontend Next.js + backend Supabase + automatización n8n + empaquetado Android vía Capacitor**.

1. Arquitectura y Stack Tecnológico

1.1 Frontend

- **Framework:** Next.js 16 (App Router) sobre **React 19**, compilación con Turbopack.
- **Lenguaje:** TypeScript (tipado estricto).
- **Estilos:** Tailwind CSS v4 (utilidades puras, tema oscuro/claro con `next-themes` y componente `ThemeToggle`).
- **Íconos:** librería `lucide-react`.
- **Gráficos:** `recharts` (barras para promedio por tipo, zona y personal).
- **Patrón de componentes:** la mayoría de las páginas son Client Components (`'use client'`) porque dependen del cliente Supabase (auth + Realtime).

1.2 Backend

- **Plataforma:** Supabase.
 - **Base de datos:** PostgreSQL con **Row Level Security (RLS)** y triggers/funciones `plpgsql`.
 - **API:** PostgREST consumida por el cliente `@supabase/supabase-js` (clave anónima de cliente, definida en variables de entorno).
 - **Autenticación:** Supabase Auth (email + contraseña), sesiones con `@supabase/ssr`.
 - **Tiempo real:** Supabase Realtime (WebSockets) sobre la publicación `supabase_realtime`.
- **Edge Functions (Deno):** una función `cambiar-password` que usa la **Service Role Key** para cambiar contraseñas de usuarios vía `auth.admin`, con validación previa de rol administrador.
- **Función SQL de respaldo:** RPC `admin_cambiar_password` (SECURITY DEFINER) como fallback cuando la Edge Function no está desplegada.

1.3 Automatización

- **n8n** (instancia cloud): dos workflows importables.
 - `workflow-sla-alerta`: recibe el evento de violación de tiempo límite del panel de limpieza e inserta una notificación.
 - `workflow-checkout-sucia`: recibe el evento de Check-Out desde recepción, marca la habitación como *Sucia* mediante PATCH a `rooms` y notifica al panel de limpieza.
 - n8n opera con la **Service Role Key** (omite RLS).

1.4 Móvil / Despliegue

- **Capacitor** (`capacitor.config.ts`, proyecto `android/`) empaqueta el build web en una app Android nativa.
- **Despliegue web:** Vercel (variable de entorno en `.env.local`, excluida del repositorio).
- **Variables de entorno clave:** URL de Supabase y clave anónima, base URL de n8n, y las dos URLs de webhooks (`NEXT_PUBLIC_N8N_WEBHOOK_URL` y `NEXT_PUBLIC_N8N_CHECKOUT_WEBHOOK_URL`).

1.5 Herramientas de desarrollo

- Dev en el puerto 3002 (`npm run dev`).
- `npm run lint` con ESLint 9 (`eslint-config-next`).
- Pruebas E2E con **Playwright** (`e2e/app.spec.ts`, navegador Chromium, base URL `localhost:3002`, variables `E2E_HOTEL_CODE` / `E2E_EMAIL` / `E2E_PASSWORD` / `E2E_ROLE` para pruebas por rol).

2. Modelo de Datos (Base de Datos en Supabase)

Todos los datos operativos residen en el esquema `public`. Convenciones: PK determinista con UUID (`gen_random_uuid()`), timestamps en `timestamptz` con valor por defecto `now()`, y aislamiento multi-tenant mediante la columna `hotel_id` en cada entidad.

2.1 Tablas del dominio operativo

Tabla	Claves y campos principales	Tipo / restricción más relevante
hotels	<code>id</code> (PK), <code>code</code> (unique), <code>name</code> , <code>created_at</code>	PK: UUID. <code>code</code> único alfanumérico (formato <code>HTL-XXXX</code>). Raíz del inquilino.
profiles	<code>id</code> (PK = FK de <code>auth.users</code>), <code>hotel_id</code> (FK → <code>hotels</code>), <code>email</code> , <code>nombre</code> , <code>role</code> , <code>activo</code> , <code>created_at</code>	PK: UUID heredado de <code>auth.users</code> ; FK de <code>hotel_id</code> . Roles operativos: <code>admin</code> , <code>repcionista</code> , <code>limpieza</code> (se acepta <code>recepcion</code> legado en login).
zonas	<code>id</code> (PK), <code>hotel_id</code> (FK → <code>hotels</code>), <code>nombre</code>	PK: UUID. Pisos/sectores del hotel (ej. "Piso 1", "Planta Baja").
rooms	<code>id</code> (PK), <code>hotel_id</code> (FK → <code>hotels</code>), <code>zona_id</code> (FK → <code>zonas</code>), <code>room_number</code> , <code>room_type</code> , <code>zone</code> (denormalizado), <code>status</code> , <code>cleaning_started_at</code> , <code>updated_at</code>	Unicidad compuesta de <code>hotel_id + room_number</code> . <code>zone</code> guarda el nombre del piso en texto para reportes y filtros. <code>cleaning_started_at</code> es el timestamp de inicio del cronómetro.
historial_estados_habitacion	<code>id</code> (PK), <code>hotel_id</code> , <code>habitacion_id</code> (FK → <code>rooms</code>), <code>usuario_id</code> (FK → <code>profiles</code>), <code>estado_anterior</code> , <code>estado_nuevo</code> , <code>fecha_cambio</code> , <code>duracion_min</code> , <code>cumplio_sla</code>	Bitácora de transiciones de estado alimentada por trigger. Índices sobre <code>fecha_cambio</code> , <code>habitacion_id</code> y <code>usuario_id</code> .

Tabla	Claves y campos principales	Tipo / restricción más relevante
room_type_config	<code>id</code> (PK), <code>hotel_id</code> (FK → hotels), <code>room_type</code> , <code>tiempo_estandar_min</code> (default 30), <code>sla_min</code> (default 45), <code>created_at</code> , <code>updated_at</code>	Unicidad compuesta <code>hotel_id + room_type</code> . Parametrización de tiempos por tipo de habitación.
notificaciones	<code>id</code> (PK), <code>hotel_id</code> (FK → hotels), <code>habitacion_id</code> (FK → rooms), <code>titulo</code> , <code>mensaje</code> , <code>tipo</code> , <code>leida</code> , <code>created_at</code>	Insertadas por n8n (Service Role); leídas/marcadas como leídas por el panel. Suscrita a Realtime.

2.2 Tablas analíticas precomputadas (mantenidas por trigger)

Tabla	Claves y campos	Restricción clave
metricas_limpieza	<code>hotel_id</code> (PK = FK → hotels), <code>total_limpiezas</code> , <code>suma_duracion_min</code> , <code>sla_cumplidas</code> , <code>sla_retrasadas</code> , <code>updated_at</code>	Resumen O(1) por hotel para los KPI del dashboard.
metricas_limpieza_detalle	<code>hotel_id</code> , <code>dimension</code> , <code>clave</code> , <code>nombre</code> , <code>limpiezas</code> , <code>suma_duracion_min</code> , <code>cumplidas</code> , <code>updated_at</code>	PK compuesta (<code>hotel_id</code> , <code>dimension</code> , <code>clave</code>). <code>dimension</code> restringida a <code>tipo</code> , <code>zona</code> , <code>personal</code> .

Nota de diseño: las tablas analíticas son **materializadas** — un trigger las actualiza de forma incremental al completarse cada limpieza, evitando consultas agregadas costosas. Se incluye un *backfill* inicial desde el historial existente.

2.3 Tablas internas de Supabase (gestionadas por la plataforma)

- **auth.users / auth.sessions / auth.identities:** credenciales y sesiones. Los metadatos custom (`hotel_id`, `role`, `nombre`, `activo`) permiten que el sistema cree automáticamente el perfil vinculado al crear usuarios desde el panel admin.

2.4 Arquitectura multi-tenant (hotel_id)

- **Principio:** cada fila de las tablas operativas y analíticas lleva `hotel_id`, que identifica al "inquilino".
- **Aplicación en la base:** políticas RLS comparan el `hotel_id` de la fila contra el `hotel_id` del perfil del usuario autenticado (`auth.uid()`). Ejemplos: lectura de `rooms`, `zonas`, `room_type_config`, `notificaciones` y `metricas_*` limitadas "al mismo hotel".
- **Aplicación en la aplicación:** todas las consultas del cliente anteponen `eq('hotel_id', )`.
- **Excepciones intencionadas:**
 - `hotels` permite `INSERT` y `SELECT` públicos para habilitar el autoregistro y el inicio de sesión por código.
 - `profiles` permite que cada usuario inserte/actualice su propio registro (`auth.uid() = id`).
 - n8n y las Edge Functions usan la **Service Role Key**, que omite RLS por diseño.

3. Mapa de Módulos y Vistas (Frontend)

3.1 Rutas públicas (sin autenticación)

/ — Landing institucional

- Presentación del producto: sincronización en tiempo real, semáforo de tiempos, diseño móvil one-touch y seguridad multi-tenant.
- Tarjetas que describen los módulos de Recepción, Limpieza y Dashboard gerencial.
- Acciones: "Registrar mi Hotel" (→ </registro>) y "Acceso al Sistema" (→ </login>).

</registro> — Alta de hotel y cuenta de administrador

- Formulario con nombre del hotel, correo del administrador y contraseña (mínimo 6 caracteres, con confirmación).
- Flujo: crea el hotel con código [HTL-XXXX](#), registra el usuario en Auth (metadatos) y crea/actualiza el perfil con rol [admin](#). Si Auth falla, elimina el hotel creado (rollback).
- Pantalla de éxito que muestra el **código del hotel** con botón de copiado y acceso directo al login con datos precargados.

</login> — Autenticación con código de hotel

- Campos: código de hotel, correo y contraseña (con selector de visibilidad y "recordarme").
- Validaciones: credenciales válidas, perfil existente, cuenta **activa** (bloquea [activo = false](#)) y coincidencia del código del hotel.
- Redirige por rol: [limpieza](#) → </limpieza>, [repcionista/repcion](#) → </repcionista>, [admin](#) → </admin>.

3.2 Módulo Administrador (</admin>)

Layout con **sidebar lateral** (Dashboard Gerencial, Gestión Paramétrica, Gestión de Usuarios), selector de tema, datos del hotel y cierre de sesión.

Vista Dashboard Gerencial

- 4 KPIs: Tiempo Promedio, Habitaciones Limpias, Alertas de Tiempo Límite (retrasadas/cumplidas) y Personal Activo.
- Gráficos de barras (recharts): Promedio por Tipo de Habitación, Rendimiento por Zona/Piso y Rendimiento por Personal, con leyendas "A Tiempo" (% min / cumplidas).
- **Tiempo real:** suscripción a [metricas_limpieza](#), [metricas_limpieza_detalle](#) y [rooms](#) que recalcula los indicadores sin recargar.

Vista Gestión Paramétrica (pestañas)

- **Zonas y Pisos:** tabla con el **nombre** del piso/zona y la **cantidad de habitaciones asociadas** (calculada por la relación con [rooms](#)). Única acción: **Modificar nombre** (para corregir tipes). No hay eliminación para preservar el historial; al renombrar se sincroniza el campo [zone](#) de las habitaciones del piso. Alta con modal en dos modos:

- **Rango numérico:** campos Desde/Hasta que generan `Piso N` en secuencia; valida la **contigüidad** (si existen del 1 al 5, el siguiente debe comenzar estrictamente en 6) y rechaza saltos o repeticiones.
- **Especial / Personalizado:** nombres libres ("Planta Baja", "PB", "Mezzanina") con validación antiduplicados.
- **Habitaciones:** tabla con número, estado operativo, zona/piso y tipo. Alta por lotes con rango (ej. 102–107), selección de zona y tipo; omite números ya existentes.
- **Tiempos/Controles:** configuración sobre `room_type_config` (tiempo estándar y **tiempo límite** por tipo de habitación), alta de nuevos tipos desde los usados en el hotel y guardado individual o masivo.

Vista Gestión de Usuarios

- Tabla del personal del hotel (avatar, nombre, correo, rol, estado). Acciones: **Modificar** (nombre, rol y contraseña vía Edge Function/RPC) y **Activar/Desactivar** acceso. Alta de nuevos usuarios vía `auth.signUp` con metadatos que crean el perfil.

3.3 Módulo Recepción (`/repcionista`)

- **Header** con badge de "Sincronización en Realtime Activa", buscador por número de habitación, campana de notificaciones y perfil del usuario.
- **Contadores por estado:** Disponibles, Ocupadas, Sucias, En Limpieza, Limpia/Lista y Mantenimiento.
- **Filtros rápidos** por cada estado + selector de zona/piso.
- **Grid de habitaciones** en tiempo real: cada tarjeta muestra número, tipo, estado con color semántico y un `select` para **cambiar el estado operativo**.
- **Cronómetro de limpieza:** mientras la habitación está "En Limpieza" muestra el tiempo transcurrido con semáforo (ámbar normal / rojo pulsante al exceder el tiempo límite) y etiqueta "Tiempo excedido".
- **Integración con n8n:** al establecer "Check-Out" se dispara el webhook que marca la habitación como Sucia y notifica al personal de limpieza.

3.4 Módulo Limpieza (`/limpieza`, móvil con barra inferior)

Layout mobile-first (máximo ancho tipo móvil, header con rol del usuario y navegación inferior de 3 pestañas). Badge "Realtime".

Pestaña Habitaciones (`/limpieza`)

- Métricas rápidas: Pendientes, Por Limpiar y En Proceso.
- **Filtro operativo combinable:** botón "Todas", pestañas dinámicas generadas desde la tabla `zonas` del hotel, y un toggle lateral "Prioritarias" **independiente** que se combina con el piso seleccionado (ej. ver el Piso 1 que además esté prioritario).
- Lista de habitaciones pendientes (Sucia / En Limpieza): tarjeta con número, tipo, zona y estado.
 - Botón **"Iniciar Limpieza"**: registra estado "En Limpieza" y `cleaning_started_at`.
 - Durante la limpieza: cronómetro en vivo, **barra de progreso** vs tiempo estándar (verde/ámbar/rojo) y etiqueta del **tiempo límite**.

- Botón "**Marcar Limpia / Lista**": finaliza, calcula duración y cumplimiento del tiempo límite, y muestra mensaje confirmatoria.
- **Alertas de tiempo límite:** al superar el límite de una limpieza en curso se notifica al webhook n8n (una única vez por habitación) para que inserte la notificación.
- **Panel de notificaciones del turno** (insertadas por n8n) con opción de marcar como leídas.

Pestaña Historial (</limpieza/historial>)

- Bitácora de limpiezas completadas: une [historial_estados_habitacion](#) con [rooms](#) y lista "Completadas hoy" (habitación, tipo, zona y hora de finalización).

Pestaña Perfil (</limpieza/perfil>)

- Tarjeta de identidad del usuario (avatar con inicial, nombre, correo, rol, hotel), estados de rol y cuenta activa, opciones de configuración (Desempeño, Notificaciones, Seguridad) y cierre de sesión.

4. Reglas de Negocio y Lógica Clave

4.1 Roles y flujo de interacción operativa

- **Recepcionista** gestiona el ciclo comercial de la habitación (Disponible/Ocupada/Check-Out, Mantenimiento).
- **Personal de Limpieza (camarera)** toma las habitaciones Sucias/En Limpieza: inicia y completa la limpieza con cronómetro; su desempeño queda registrado por [usuario_id](#) en el historial y se traduce en métricas por personal.
- **Administrador** parametriza zonas, habitaciones y tiempos; activa/desactiva usuarios; monitorea KPIs y cumplimiento de tiempos.

4.2 Ciclo de estados de la habitación y RN-01

Cadena canónica:

Disponible → Ocupada → Check-Out → Sucia → En Limpieza → Limpia/Lista → Disponible (con **Mantenimiento** como estado lateral).

- **RN-01 (inmutabilidad de transición):** un trigger impide pasar una habitación a "Disponible" desde cualquier estado intermedio; debe completar antes el ciclo Check-Out → Limpia/Lista. Esto evita asignar habitaciones no verificadas.
- **Automatización post Check-Out (RF-04):** al registrar Check-Out, recepción dispara el webhook n8n que marca la habitación como "Sucia" y encola la tarea de limpieza, además de notificar al panel.

4.3 Sincronización en tiempo real (Supabase Realtime)

- **Suscripciones activas:** estado de habitaciones ([rooms](#)), notificaciones ([notificaciones](#)) y métricas gerenciales ([metricas_limpieza](#) y [metricas_limpieza_detalle](#)).

- **Comportamiento:** cualquier `INSERT/UPDATE/DELETE` en esas tablas se propaga por WebSocket a los paneles conectados (limpieza, recepción y dashboard) sin recargar.
- **Detalle técnico:** las tablas de métricas usan `REPLICA IDENTITY FULL` para que los eventos de escritura incluyan el `hotel_id` y el filtro de suscripción por hotel funcione correctamente.

4.4 Tiempos operativos y estándares

- **Parametrización por hotel y tipo de habitación** en `room_type_config`.
 - **Tiempo estándar** (`tiempo_estandar_min`, por defecto 30 min): referencia de la barra de progreso y del criterio "**Prioritarias**" (una habitación en limpieza se vuelve prioritaria al superarlo).
 - **Tiempo límite** (`sla_min`, por defecto 45 min): umbral de **alerta crítica** (rojo), dispara la notificación al webhook y define el "cumplimiento a tiempo".
- **Semáforo:** verde (dentro de estándar) → ámbar (superó estándar, aún dentro de límite) → rojo crítico (superó el límite).
- **Cálculo del cumplimiento:** al completar la limpieza, el trigger del historial calcula `duracion_min = ahora - inicio` y `cumplio_sla = duración ≤ límite del tipo`.

4.5 Métricas y reportes (lógica analítica)

- El trigger de historial registra cada transición; el trigger de métricas actualiza de forma **incremental y O(1)** el resumen del hotel y los desgloses por `tipo` de habitación, `zona`/piso y `personal`.
- Solo las limpiezas **completadas** alimentan métricas (no cuentan estados intermedios).
- El dashboard mantiene los KPI y gráficos frescos vía Realtime sobre las tablas resumen; se provee un *backfill* inicial para migrar el historial existente.

4.6 Autenticación, seguridad y multitenant

- Ingreso con contraseña de Supabase Auth + **código de hotel** obligatorio en el login.
- Una cuenta con `activo = false` **no puede ingresar** (se deniega y cierra la sesión).
- Todo el acceso a datos pasa por **RLS por hotel**; la única vía con privilegios ampliados son las Edge Functions y n8n (Service Role).
- **Cambio de contraseña:** únicamente un administrador puede hacerlo, mediante la Edge Function (con `auth.admin`) y, en caso de fallo, el RPC de base de datos `admin_cambiar_password` (ambos validan el rol antes de ejecutar).

4.7 Alta de hoteles y usuarios

- El registro genera un código único `HTL-XXXX`; el administrador se crea vía Auth con metadatos y se garantiza la existencia del perfil (con `role = admin` y `activo = true`).
- Al crear usuarios desde el panel admin, los metadatos de `signUp` permiten que el perfil se genere automáticamente con el rol elegido.